

7.3 - Retry, timeout e backoff

Os padrões que absorvem falhas em vez de amplificá-las

Retry sem freio derruba dois serviços

- **Serviço de notificação chama a API de SMS**
 - API fica lenta e as retentativas se acumulam

LUNALOG

O serviço de notificação da LunaLog chamava a API de SMS de forma síncrona. Quando essa API ficou lenta, o serviço começou a retentar imediatamente, três vezes por falha, sem nenhum intervalo entre as tentativas. Em 10 minutos havia 8.000 requisições pendentes em memória, o heap esgotou e o serviço caiu. Junto com ele foi o serviço de confirmação de entrega, que o chamava de forma síncrona. Uma lentidão numa dependência virou cascata por ausência de padrões básicos.

Retry, timeout e backoff trabalham juntos

Cada um cobre uma lacuna do outro

- **Retry só faz sentido para erros transientes**
 - nunca retentar erro de validação ou de autorização
- **Timeout impede o acúmulo de requisições pendentes**
 - sem ele, o retry sozinho enche a memória
- **Backoff exponencial aumenta o intervalo entre tentativas**
 - evita martelar uma dependência que já está sofrendo
- **Jitter espalha as retentativas ao longo do tempo**
 - previne o thundering herd de todos retentando juntos

Backoff exponencial com jitter

PYTHON

```
import random, time

def call_with_retry(fn, max_tries=4, base=0.5, cap=8.0):
    for attempt in range(max_tries):
        try:
            return fn(timeout=2.0) # timeout sempre presente
        except TransientError:
            if attempt == max_tries - 1:
                raise
            delay = min(cap, base * (2 ** attempt))
            delay = delay * (0.5 + random.random()) # jitter
            time.sleep(delay)
```

Retry sem timeout é uma armadilha

- Retentar sem intervalo amplifica a sobrecarga
- Retentar sem timeout acumula requisições até o OOM

Retry isolado é perigoso. Sem timeout, cada tentativa fica presa esperando uma resposta que não vem, e a memória enche. Sem backoff, você bombardeia uma dependência já fragilizada. Os três padrões só protegem quando aplicados juntos, exatamente o que faltava no serviço de notificação da LunaLog.

- Calibre retry apenas para o que pode dar certo na repetição



Retry, timeout e backoff seguram falhas transientes e rápidas. Mas e quando a dependência simplesmente não volta? Quando o backoff já esgotou e o serviço externo continua fora por minutos a fio? Negar tudo ao usuário é a única saída? Existe uma resposta melhor do que o erro completo: entregar menos, de propósito, e manter o sistema de pé.

Fallback gracioso e graceful degradation